

Contents

I Code Reference

1	Bitmasksubsets	1
2	Dsu	1
3	Lis	1
4	Segtree	1
5	Toposort	2

II Theoretical Appendix

1	Lis	2
2	Permutations	2
3	Segtree	2
4	Toposort	2

Part I

Code Reference

1 Bitmasksubsets

```
827 for (int mask = 0; mask < (1 << n); mask++) {
e0a     for (int submask = mask; submask != 0; submask = (
        submask - 1) & mask) {
982         int subset = mask ^ submask;
            // do whatever you need to do here
70a     }
e71 }
```

2 Dsu

```
6a9 class DSU{
673     public:
1a8         int n;
f92         vector<int> p, r, s;

ea1     DSU(int nodes): p (nodes), r (nodes, 1), s (nodes,
1){
b67         n = nodes;
9e4         for (int i=0; i<n; i++) p[i] = i;
dc3     }
/*
ac5     ll find(ll x){
eab         return (p[x] == x? x : p[x] = find(p[x]));
07f     }
c4c     */
57b     int find(int x){
42e         int root=x;
39f         while (root != p[root]) root = p[root];
133         int aux;
016         while (x != p[x]){
615             aux = p[x];
88a             p[x] = root;
835             x = aux;
8db         }
ea5         return x;
eb6     }

2e7     bool unite(int x, int y){
f08         x = find(x), y = find(y);
e65         if (x == y) return false;
7ce         if (r[x] > r[y]) swap(x, y);
b6a         if (r[x] == r[y]) r[y]++;
d16         s[y] += s[x];
ae9         p[x] = y;
15a         n--;
8a6         return true;
c37     }

082 };
```

3 Lis

```
f16 int find_lis(const vector<int> &a) {
471     vector<int> dp;
941     for (int i : a) {
01d         int pos = lower_bound(dp.begin(), dp.end(), i) - dp
        .begin();
56a         if (pos == dp.size()) dp.push_back(i);
d2f         else dp[pos] = i;
            // we can have a new, longer subsequence or
            // we can make the ending element smaller
7af     }
269     return dp.size();
b75 }
```

4 Segtree

```
853 template <class T> class Segtree{
673     public:
1a8         int n;
53f         T neutral;
22c         vector<T> t;
8c2         Segtree(int size, T neutral): neutral(neutral){
ba7             n = 1;
321             while (size > n) n <<= 1;
7ce             t.resize(2*n);
62b         }

1cb         T op(T a, T b){
```

```

534         return a + b;
4ed     }

6ab     T query(int l, int r, int idx, int a, int b){
41a         if (r < a || b < l) return neutral;
877         if (l <= a && b <= r) return t[idx];
e26         int m = (a+b)/2;
0f2         return op(query(l, r, 2*idx, a, m), query(l, r,
2*idx+1, m+1, b));
fd6     }
190     T query(int l, int r){ return query(l, r, 1, 0, n
-1); }

671     void add(int idx, T v){
aab         if (idx < 0 || idx >= n) return;
cae         t[idx += n] += v; // aqui era pra ser op() invê
s de +=
4ef         for (idx >= 1; idx; idx >= 1) t[idx] = op(t
[2*idx], t[2*idx+1]);
7ef     }
fdf };

```

5 Toposort

Retorna as componentes na ordenação topológica reversa. O número de componentes é $ncomp$. Talvez o certo seja $\min(low[u], num[v])$, mas vou usar assim até achar um erro. Estado em *vis*://

- 0: não visitado
- 1: finalizado
- 2: ativo

```

e3b vector<int> g [mx], comp (mx), low (mx), num (mx), vis
(mx);
82b stack<int> vert;
22d int ti=0;
c02 int ncomp=0;
e0e void tarzan(int u){
873     low[u] = num[u] = ++ti;
10a     vis[u] = 2; // active
7a5     vert.push(u);

7b9     for (auto v: g[u]){
d90         if (!vis[v]){
5c1             tarzan(v);
ab6             low[u] = min(low[u], low[v]);
555         }
2d3         if (vis[v] == 2) low[u] = min(low[u], low[v]);
// talvez o certo seja min(low[u], num[v]) mas vou
usar assim até achar um erro
dd6     }

382     if (num[u] == low[u]){
a8f         ncomp++;
690         while (!vert.empty()){
08a             int v = vert.top(); vert.pop();
d6f             comp[v] = ncomp;
cca             vis[v] = 1;
163             if (u == v) break;
287         }
734     }
a7f }

```

Part II

Theoretical Appendix

1 Lis

- Encontrar a LIS de um array reverso é equivalente a encontrar a **LDS** (Longest Decreasing Subsequence).
- Necessita de adaptação para *longest non-increasing subsequence*. Para restaurar a subsequência, use um array auxiliar (vetor de pais).
- **Patience sorting**: Pensar no problema como formar pilhas, colocando o elemento atual na pilha com o menor

topo que seja maior que ele. (Útil para modelagem geométrica ou de restrições).

- O número de LIS distintas pode ser calculado em $\mathcal{O}(N \log N)$ com SegTree.
- **LCS em Permutações**: A primeira permutação define o índice alvo para cada valor. Você substitui os valores da segunda permutação pelos índices correspondentes da primeira. A tarefa se reduz a achar a LIS do novo array, já que cada elemento aparece exatamente uma vez.

2 Permutations

- O código para calcular a paridade da permutação está no caderno.
- **Face Value**: Use as definições de permutação de forma literal. Uma permutação de tamanho N consiste em inteiros estritamente distintos de 1 a N .
- **Divisibilidade**: Devido à limitação de limite, existem no máximo $\lfloor N/k \rfloor$ elementos na permutação que são múltiplos de k .
- **Validação Rápida**: Como o multiset de uma permutação válida de tamanho M é sempre único $\{1, 2, \dots, M\}$, uma abordagem direta para checar se um subarray forma uma permutação é comparar os multisets utilizando algoritmos de **Hashing**.

3 Segtree

- Lembre-se de usar **compressão de coordenadas/ pontos** se o domínio for muito grande. Alguns problemas exigem manter duas SegTrees simultâneas.
- Funciona para qualquer operação **associativa**.
- É comum utilizar operações **não-comutativas** (ex: manter a soma do prefixo, sufixo e valor total de um subarray). Tenha atenção especial na função de *merge*: a operação com o filho esquerdo pode diferir do direito, e a ordem importa estritamente.
- **Range update sem Lazy**: É possível fazer atualizações em intervalo se você tratar a SegTree como um *difference array* (array de diferenças) e pedir a soma de prefixo para a consulta.

4 Toposort

- **Algoritmo de Kahn**: O código padrão (por graus de entrada) está no caderno.
- Aparece frequentemente em problemas que não parecem ser de grafos à primeira vista. Basta modelar as dependências como um **DAG** (relações estritas de "vem antes / depois").
- A ordenação topológica permite aplicar **Programação Dinâmica (DP)** processando os nós linearmente.
- **Via DFS**: Adicione o nó a uma lista ao final da chamada recursiva e, no término, reverta a lista.
- **Validação (Detecção de Ciclos)**: É mandatório checar se a ordenação resultante é válida. Para toda aresta $U \rightarrow V$, se $ind[V] \leq ind[U]$ na ordenação final, existe um ciclo e o toposort é inválido.